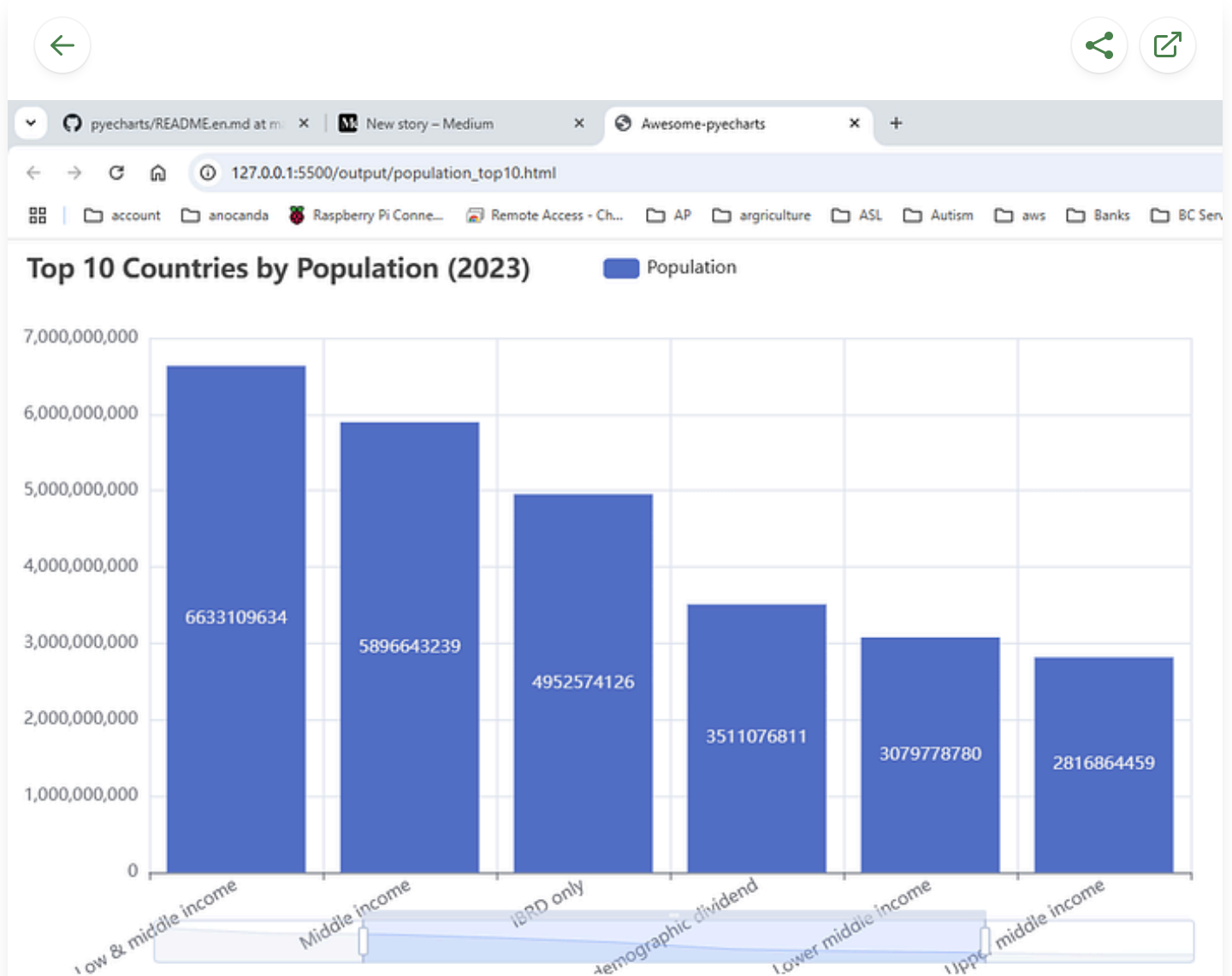




July 5, 2025

Visualizing Data with Pyecharts: A Complete Tutorial with Real-World Examples

From Raw Data to Interactive Visuals with Pyecharts and Python

**By Dr. Shouke Wei**

4 min read

[Download article](#) ▾**Contents** 26 ▾

1 Table of Contents:

2 🧭 Introduction

3 ❓ Why Use Pyecharts?

4 📦 Installation

5 📁 Real Data Source

[Show all 26 sections](#)

A Step-by-Step Guide to Beautiful Python Visualizations with Pyecharts

Table of Contents:

• 🧭 [Introduction](#) • ❓ [Why Use Pyecharts?](#) • 📦 [Installation](#) • 📁 [Real Data Source](#) • 📈 1. [Line Chart: Global CO₂ Emissions](#) • 📁 [Data Source](#) • 📊 [Visualization with Pyecharts](#) • 📊 2. [Bar Chart: Top 10 Countries by Population \(2023\)](#) • 📁 [Data Source](#) • 📊 [Visualization](#) • 📍 3. [Map Chart: Global GDP by Country](#) • 📁 [Data Source](#) • 🌐 [Visualization](#) • 🌈 4. [Pie Chart: Browser Market Share](#) • 📁 [Real-time Example](#) • ✅ [Automatically open a chart](#) • ✅ [Just add this after .render\(\):](#) • 📌 [Full Example](#) • 📝 [Notes](#) • ✅ [render_and_open Function](#) • 📦 [Example Usage](#) • 🛠️ [Optional: Add to Utility Module](#) • ⭐ [Summary](#) • 📁 [Bonus: How to View Charts](#) • 🔗 [References](#)

🧭 Introduction

Pyecharts is a powerful and flexible data visualization library in Python, built on top of the JavaScript charting library [ECharts](#). It allows you to create stunning interactive visualizations that can be rendered in Jupyter notebooks, web pages, and more — all using simple Python code.

Key Features:

- Rich chart types: bar, line, pie, map, word cloud, gauge, etc.
- Interactive and customizable
- Easy integration with Flask, Django, and Jupyter
- Exports to HTML, PNG, and SVG

? Why Use Pyecharts?

- **Interactive by default:** Unlike Matplotlib, charts are interactive (tooltips, zoom, hover).
- **Beautiful designs:** Charts look modern and clean, thanks to ECharts.
- **Web-friendly:** Easily export to HTML or embed in web apps.
- **Chinese and international maps support:** Great for geospatial visualizations.
- **Easy API:** Clear, object-oriented Python interface.

Installation

pip install:

```
1 pip install pyecharts
```

Install from source:

```
1 $ git clone https://github.com/pyecharts/pyecharts.git
2 $ cd pyecharts
3 $ pip install -r requirements.txt
4 $ python setup.py install
```

For map support:

```
1 pip install echarts-countries-pypkg
```

Real Data Source

Let's fetch real-world data from:

- [GitHub's Raw CSV File](#) 
- [Kaggle](#) 
- [World Bank Open Data](#) 
- [Our World in Data](#) 

1. Line Chart: Global CO₂ Emissions

Data Source

From **Our World in Data**:

```
1 import pandas as pd
2
3 url = "https://raw.githubusercontent.com/owid/co2-data/master/owid-co2-da
4 df = pd.read_csv(url)
5 # Filter for world data
6 world_df = df[df['country'] == 'World']
7 world_df = world_df[['year', 'co2']].dropna()
```

Visualization with Pyecharts

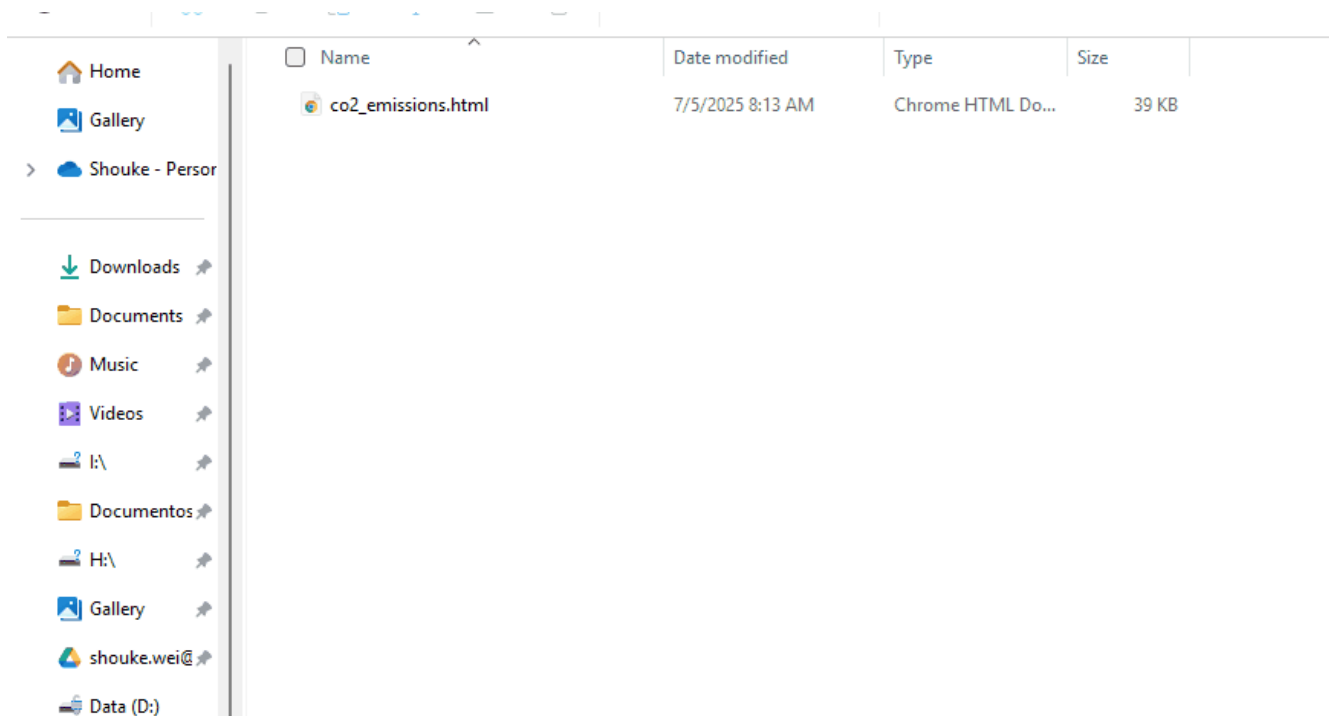
```
1 from pyecharts.charts import Line
2 from pyecharts import options as opts
3
4 line = (
5     Line()
```

```

6     .add_xaxis(world_df['year'].astype(str).tolist())
7     .add_yaxis("CO2 Emissions (billion tonnes)", world_df['co2'].round(2))
8     .set_global_opts(
9         title_opts=opts.TitleOpts(title="Global CO2 Emissions Over Time"),
10        tooltip_opts=opts.TooltipOpts(trigger="axis"),
11        yaxis_opts=opts.AxisOpts(name="CO2 (Gt)"),
12        xaxis_opts=opts.AxisOpts(name="Year")
13    )
14 )
15 line.render("./output/co2_emissions.html")

```

You can save in any path, here it is saved in the " ./output/co2_emissions.html. " in the project folder. Then you can open the chart with your favorite browser:



2. Bar Chart: Top 10 Countries by Population (2023)

Data Source

From GitHub (pre-cleaned version):

```

1 url = "https://raw.githubusercontent.com/datasets/population/master/
2 df = pd.read_csv(url)

```

```
3
4 latest = df[df['Year'] == 2023]
5 top10 = latest.sort_values(by='Value', ascending=False).head(10)
```

Visualization

```
1 from pyecharts.charts import Bar
2 from pyecharts import options as opts
3
4 bar = (
5     Bar()
6     .add_xaxis(top10['Country Name'].tolist())
7     .add_yaxis("Population", top10['Value'].tolist())
8     .set_global_opts(
9         title_opts=opts.TitleOpts(title="Top 10 Countries by Population (
10         xaxis_opts=opts.AxisOpts(axislabel_opts=opts.LabelOpts(rotate=30)
11         datazoom_opts=opts.DataZoomOpts(),
12     )
13 )
14 bar.render("./output/population_top10.html")
```

3. Map Chart: Global GDP by Country

Data Source

Use a cleaned World Bank GDP dataset:

```
1 gdp_data = {
2     "United States": 26344,
3     "China": 17938,
4     "Japan": 4206,
5     "Germany": 4223,
6     "India": 3416,
7     "United Kingdom": 3305,
8     "France": 3061,
9     "Italy": 2400,
10    "Brazil": 2113,
```

```
11     "Canada": 2253,  
12 }
```



Visualization

```
1 from pyecharts.charts import Map  
2 from pyecharts import options as opts  
3  
4 map_chart = (  
5     Map()  
6     .add("GDP (in billion USD)", [list(z) for z in gdp_data.items()], "wo  
7     .set_global_opts(  
8         title_opts=opts.TitleOpts(title="World GDP by Country"),  
9         visualmap_opts=opts.VisualMapOpts(max_=27000),  
10    )  
11 )  
12 map_chart.render("./output/world_gdp_map.html")
```



4. Pie Chart: Browser Market Share



Real-time Example

```
1 from pyecharts.charts import Pie  
2 from pyecharts import options as opts  
3  
4 data = [  
5     ("Chrome", 65.5),  
6     ("Safari", 18.7),  
7     ("Edge", 4.2),  
8     ("Firefox", 3.4),  
9     ("Others", 8.2),  
10 ]  
11  
12 pie = (  
13     Pie()  
14     .add("", data)  
15     .set_global_opts(title_opts=opts.TitleOpts(title="Browser Market Shar
```

```
16     .set_series_opts(label_opts=opts.LabelOpts(formatter="{b}: {d}%"))
17 )
18 pie.render("./output/browser_share.html")
```

✅ Automatically open a chart

When you create visualizations with **Pyecharts**, charts are rendered as `.html` files. To make the workflow smoother, you can **automatically open** the chart in your default web browser right after rendering. To **automatically open a Pyecharts chart in the default web browser** after rendering the `.html` file, you can use Python's built-in `webbrowser` module.

✅ Just add this after `.render()` :

```
1 import webbrowser
2 import os
3
4 file_path = os.path.abspath("your_chart.html")
5 webbrowser.open(f"file://{file_path}")
```

📌 Full Example

```
1 from pyecharts.charts import Pie
2 from pyecharts import options as opts
3 import webbrowser
4 import os
5
6 # Create a simple pie chart
7 data = [("Chrome", 65.5), ("Safari", 18.7), ("Edge", 4.2), ("Firefox", 3.
8 pie = (
9     Pie()
10     .add("", data)
11     .set_global_opts(title_opts=opts.TitleOpts(title="Browser Market Shar
12     .set_series_opts(label_opts=opts.LabelOpts(formatter="{b}: {d}%"))
13 )
14 # Render to HTML
```



```

15 output_file = "./output/browser_share.html"
16 pie.render(output_file)
17 # Automatically open in default browser
18 file_path = os.path.abspath(output_file)
19 webbrowser.open(f"file://{file_path}")

```



Notes

- `os.path.abspath()` ensures the full file path is used.
- `webbrowser.open()` launches the default browser with the file.



render_and_open Function

In this tutorial, we save the charts in the `./output/` directory as an example. You can create the **reusable function** that can be used with any Pyecharts chart to render and open automatically in your default web browser.

```

1  import os
2  import webbrowser
3
4  def render_and_open(chart, filename: str = "chart.html"):
5      """
6      Renders a Pyecharts chart to ./output/filename and opens it in the de
7
8      Args:
9          chart (Chart): A Pyecharts chart instance.
10         filename (str): Output HTML filename (default: "chart.html").
11     """
12     output_dir = "output"
13     os.makedirs(output_dir, exist_ok=True) # Ensure output directory exi
14     output_path = os.path.join(output_dir, filename)
15
16     chart.render(output_path)
17     file_path = os.path.abspath(output_path)
18     webbrowser.open(f"file://{file_path}")

```



Example Usage

```

1 from pyecharts.charts import Bar
2 from pyecharts import options as opts
3
4 # Sample bar chart
5 bar = (
6     Bar()
7     .add_xaxis(["A", "B", "C", "D"])
8     .add_yaxis("Example", [10, 20, 30, 40])
9     .set_global_opts(title_opts=opts.TitleOpts(title="Sample Bar Chart"))
10 )
11 # Use the reusable function
12 render_and_open(bar, "browser_share.html") # Opens ./output/browser_shar

```

Since our reusable `render_and_open()` function in this tutorial already appends the `./output/` directory internally, you should not include `./output/` again in the filename when calling it.

Otherwise, the path becomes `./output/./output/sample_bar_chart.html`, which is redundant and might cause issues.

Optional: Add to Utility Module

You can save the function in a `utils.py` file and import it anywhere:

```

1 # utils.py
2 def render_and_open(chart, filename="chart.html"):
3     ...

```

Then use it:

```

1 from utils import render_and_open
2 render_and_open(my_chart)

```

Summary

Pyecharts is a **versatile, web-friendly, and elegant** charting library for anyone needing visual insights in Python. Here's a quick recap of what we covered:

Chart Type	Use Case Example	File Output
Line	Time series like CO ₂ emissions	<code>co2_emissions.html</code>
Bar	Top categories (population, sales)	<code>population_top10.html</code>
Map	Country-wise statistics (GDP, cases)	<code>world_gdp_map.html</code>
Pie	Percentage shares (browser, market)	<code>browser_share.html</code>

Bonus: How to View Charts

1. All charts are rendered as standalone **HTML files**.
2. You can open them in any browser.
3. To integrate into a dashboard: embed the HTML using `<iframe>` or render inside a web app (Flask, Streamlit, etc.).
4. You can use Python's built-in `webbrowser` module to instant preview of your visualizations.
5. Define a **reusable function** to open charts automatically.

References

- [Pyecharts Documentation](#) 
- [ECharts Official Site](#) 
- [Our World in Data — CO₂ Data](#) 
- [World Bank Open Data](#) 
- [Datasets on GitHub](#) 



